

Agentic AI and Workflow Automation

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité

<https://github.com/mohamedYoussfi/agentic-ai>

<https://www.youtube.com/watch?v=ev09WsppHE>



NetAcad SUMMER CAMP 2025

E-LEARNING COURS EN LIGNE

AOÛT ET SEPTEMBRE

S'INSCRIRE ICI

Cybersecurity & Networking

- Introduction to Cybersecurity
- Cyberthreat Management
- CCNA: Introduction to Networks
- Ethical Hacker

Programming, AI & Digital Innovation

- Python Essentials I
- HTML Essentials
- Discovering Entrepreneurship
- Introduction to Modern AI

Networking CISCO Academy

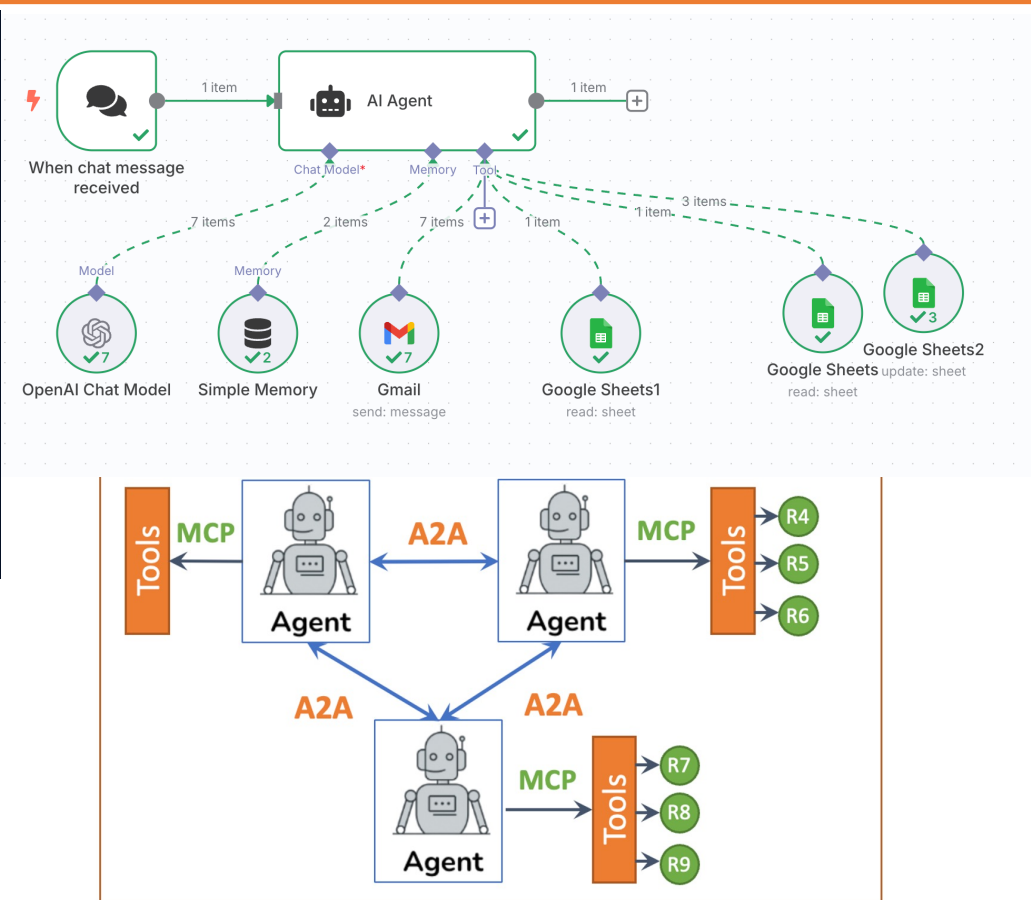
Networking CISCO Academy

AGENTIC AI AND WORKFLOW AUTOMATION

Avec Prof. Mohammed YOUSSEFI

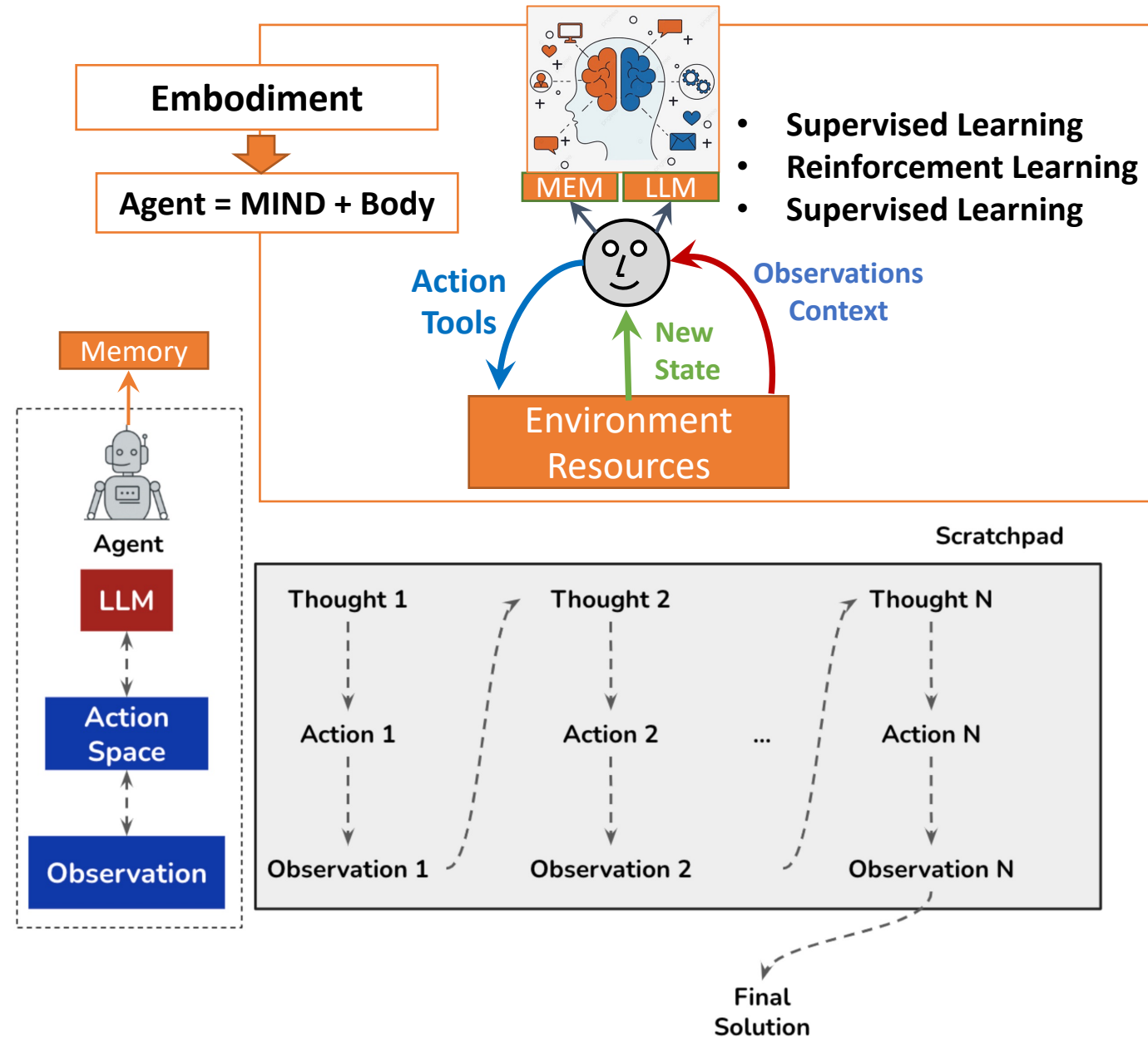
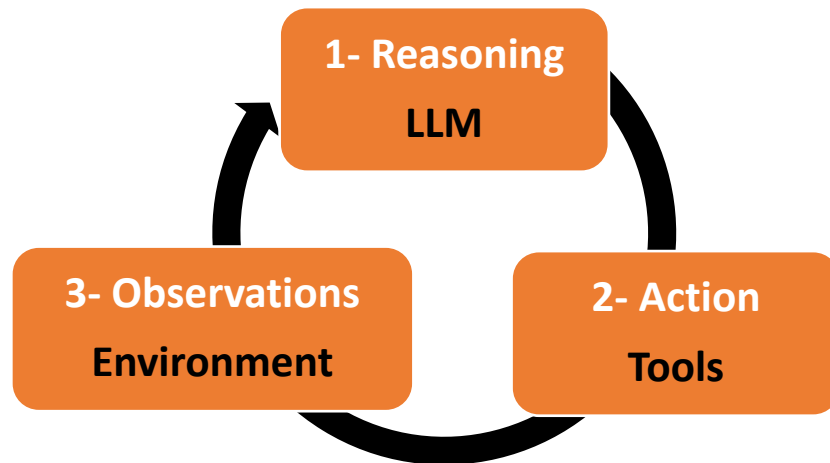
Rejoindre en ligne le webinaire

VENDREDI 12 septembre 2025
16 h 00 - 18 h 00 GMT +1



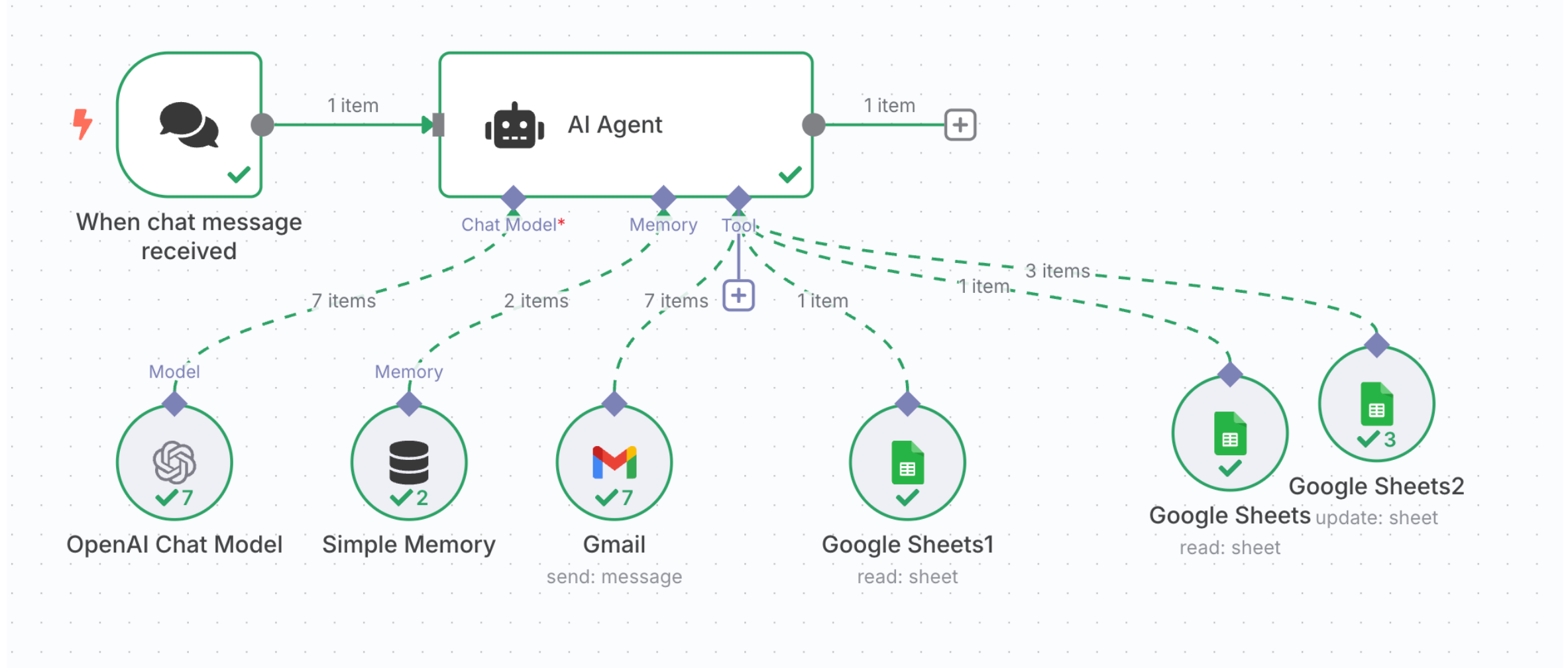
AI Agents

- An AI Agent, as described by the **ReaAct** Framework, is an **autonomous** entity with a goal that can:
 - **Reason** based on context (environmental abstraction)
 - **Act autonomously** on its environment
 - **Collect observations** about the environment



Demo 1 : First Agent in Automation platform : n8n

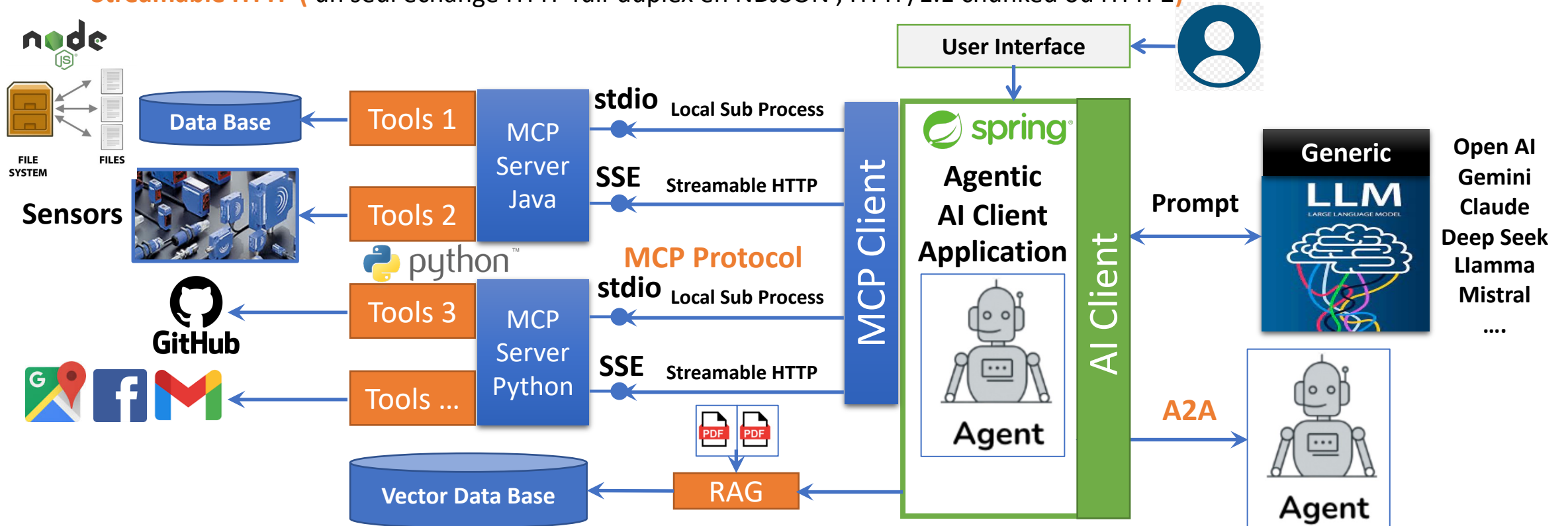
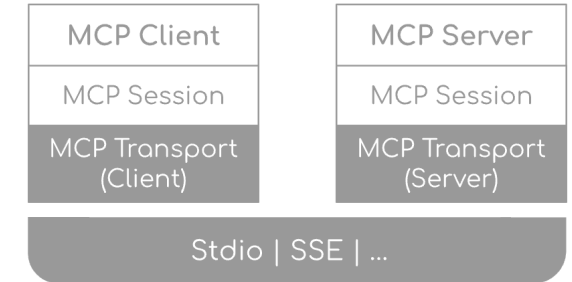
<https://www.youtube.com/watch?v=E0OZPy5fP8E>



Model Context Protocol (MCP)

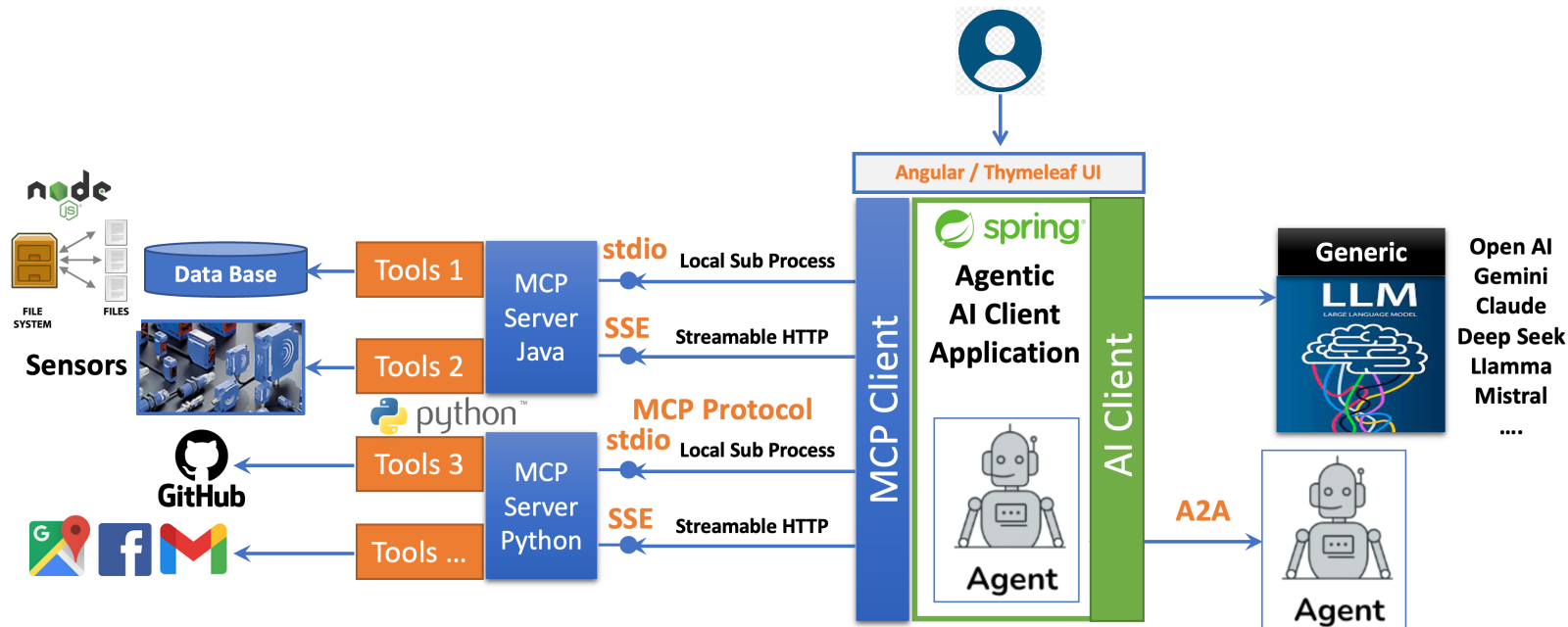
<https://modelcontextprotocol.io>

- MCP standardizes how to integrate **additional context and tools** into the ecosystem of AI applications
- The protocol uses **JSON-RPC 2.0 messages**
- The protocol defines two standard transport mechanisms for client-server communication:
 - **STDIO**, communication over **standard input** and **standard output**
 - **SSE (Server Sent Event)** : serveur → client en streaming + POST séparés pour client → serveur
 - **Streamable HTTP** (un seul échange HTTP full-duplex en NDJSON , HTTP/1.1 chunked ou HTTP2)

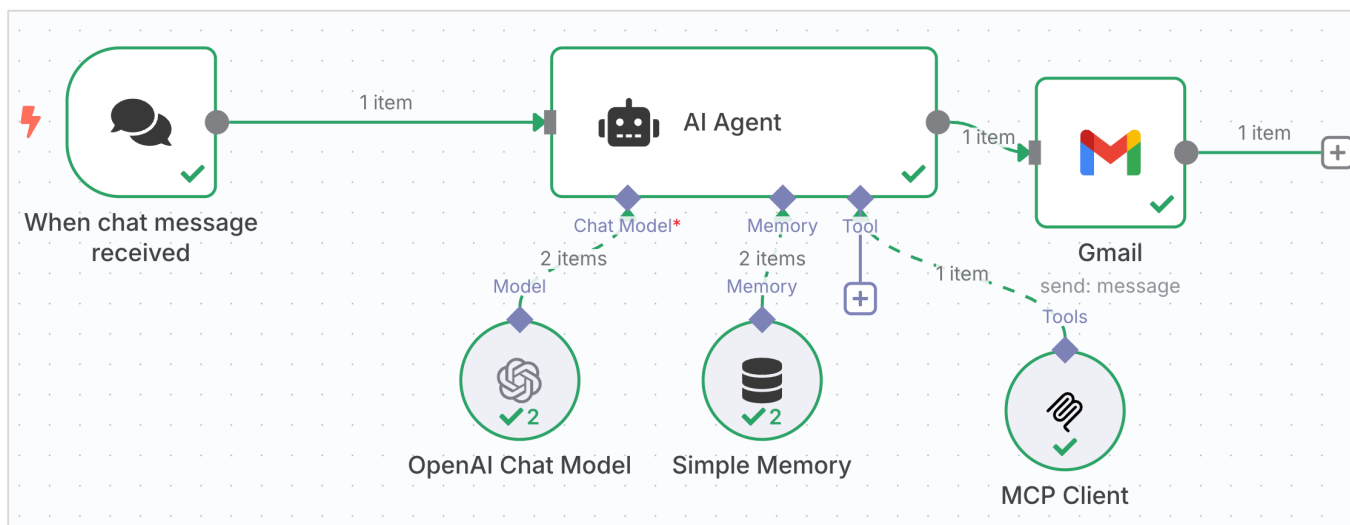


MCP

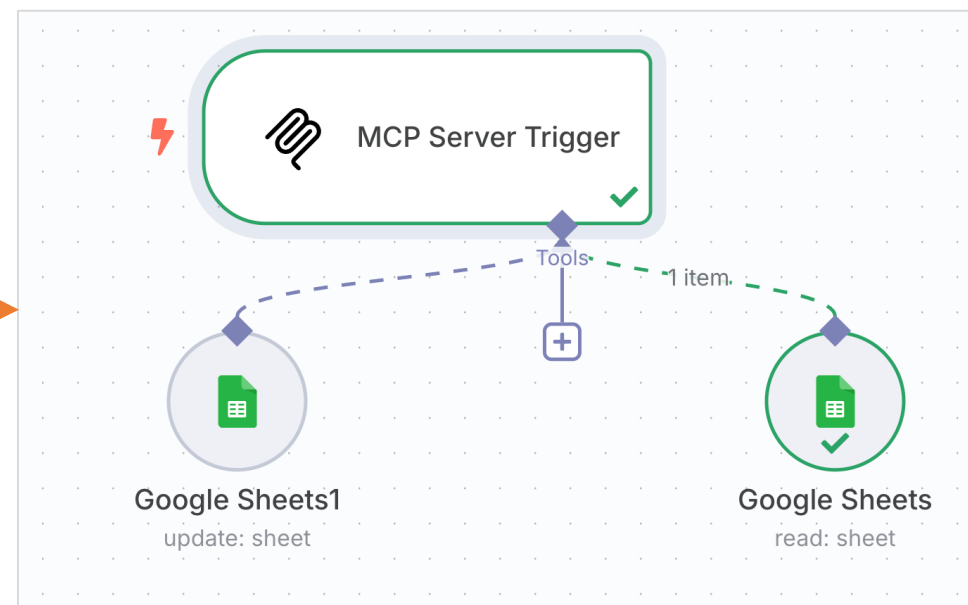
- Model Context Protocol (MCP) is an open protocol that enables seamless integration between LLM applications and external data sources and tools.
- MCP provides a standardized way for applications to:
 - Share contextual information with language models
 - Expose tools and capabilities to AI systems
 - Build composable integrations and workflows
- The protocol uses **JSON-RPC 2.0 messages** to establish communication between:
 - **Hosts:** LLM applications that initiate connections
 - **Clients:** Connectors within the host application
 - **Servers:** Services that provide context, tools and capabilities
- MCP standardizes how to integrate **additional context and tools** into the ecosystem of AI applications.



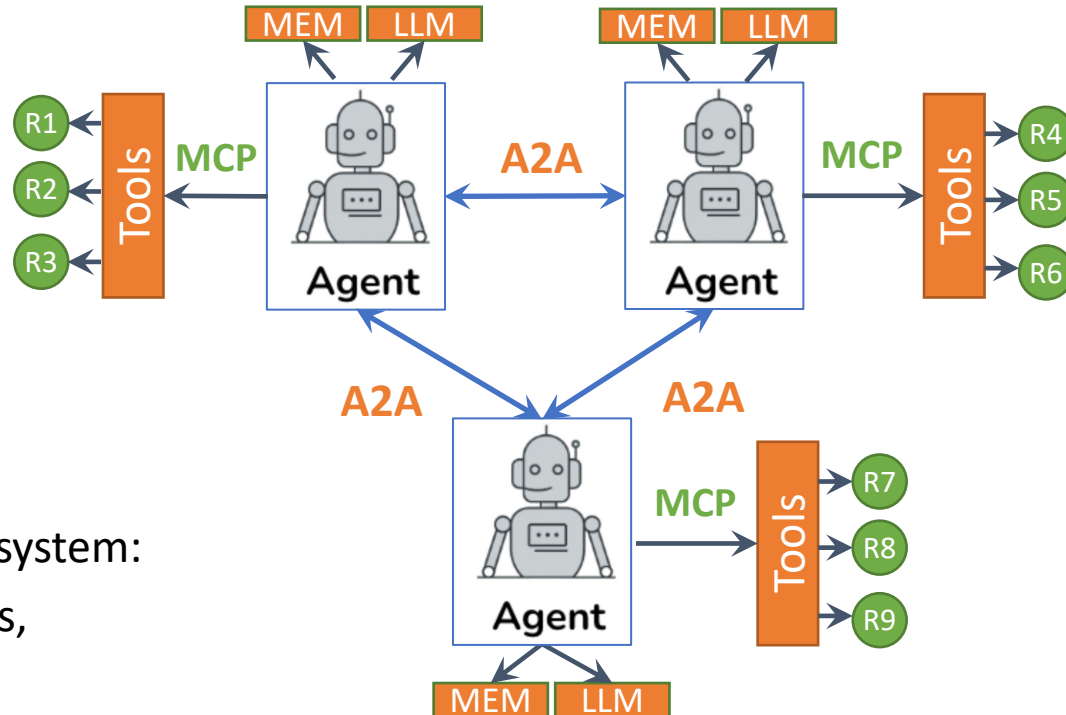
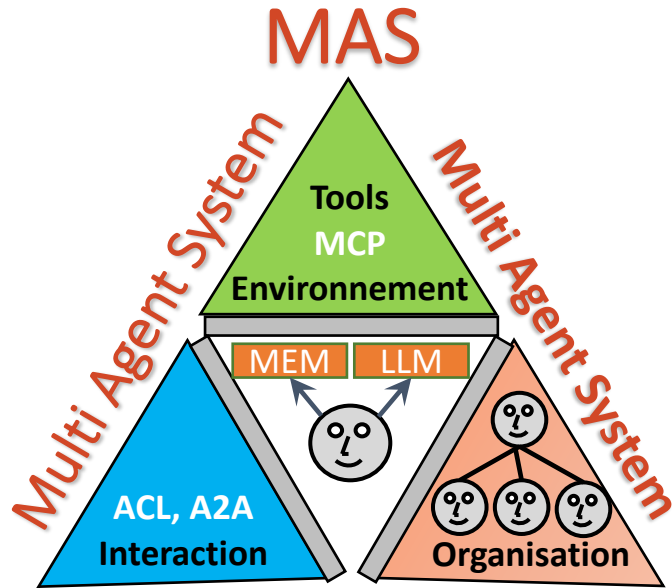
Demo 2 : AI Agent with MCP



MCP

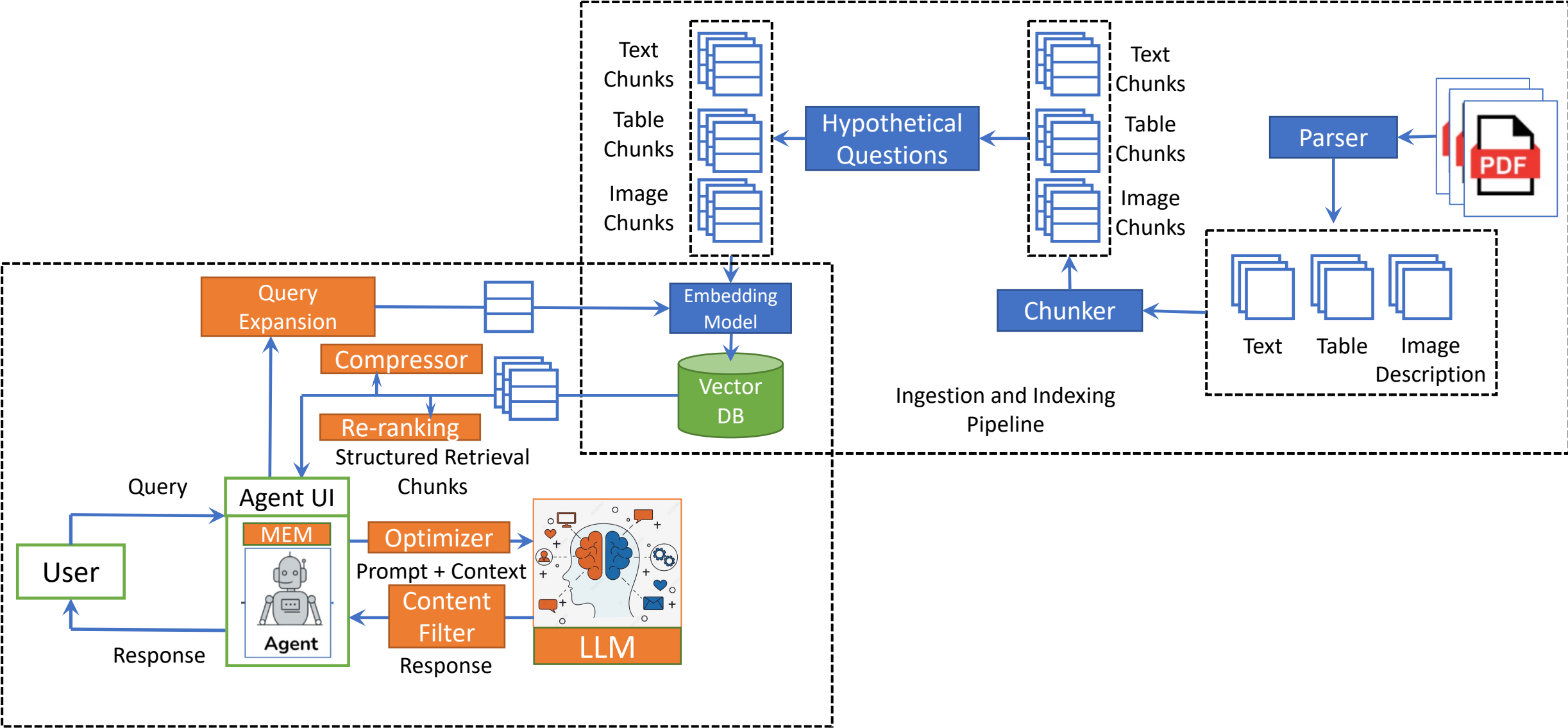


MAS And new Protocols : MCP et A2A

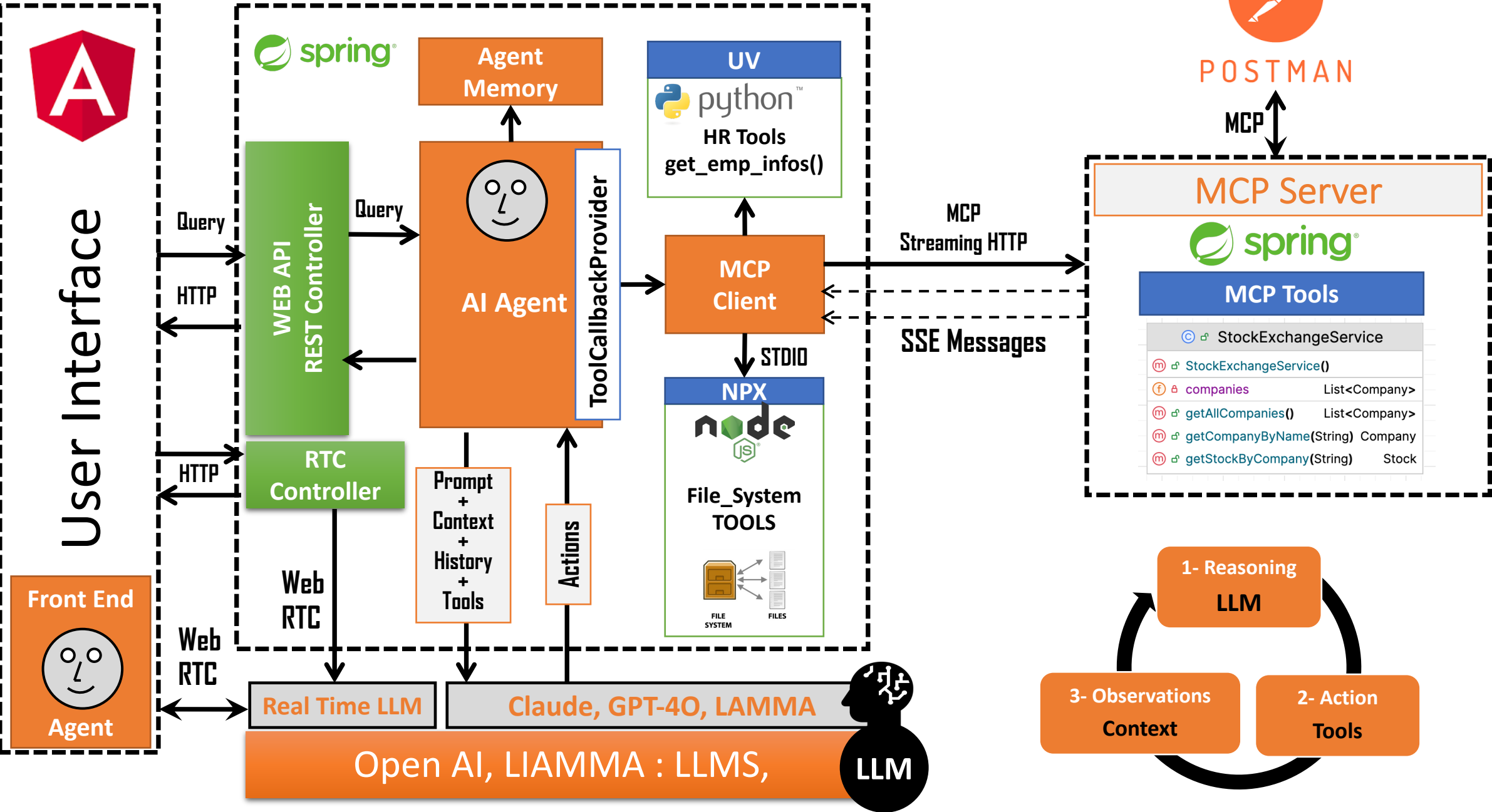


- A multi-agent system (MAS) is a distributed system:
 - Composed of a set of distributed agents,
 - Located in a specific environment
 - and Interacting according to specific organizational structures.
- An MAS solves complex problems by leveraging the collective intelligence of its component agents (DAI).

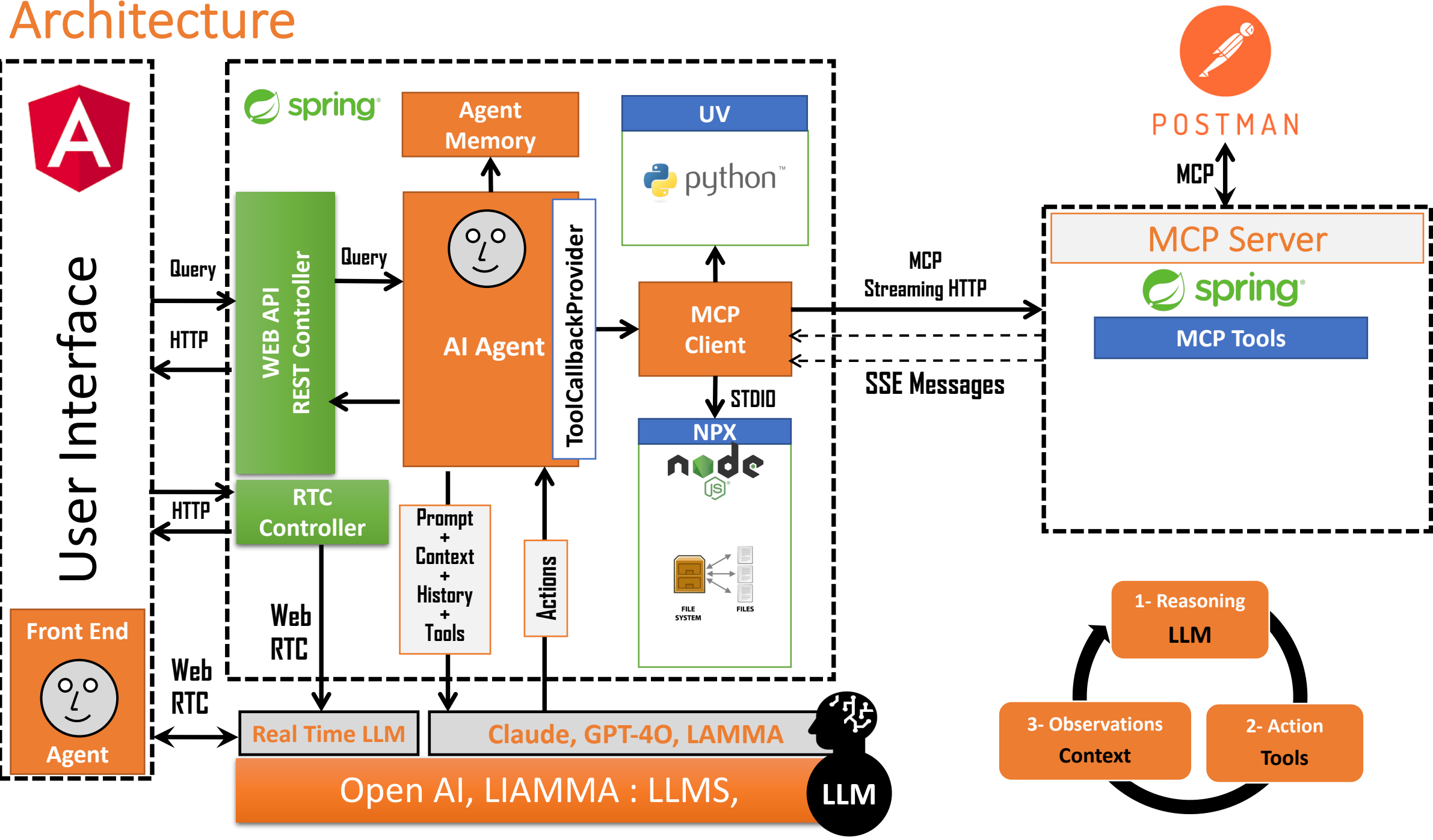
Advanced Multi Modal Agentic RAG



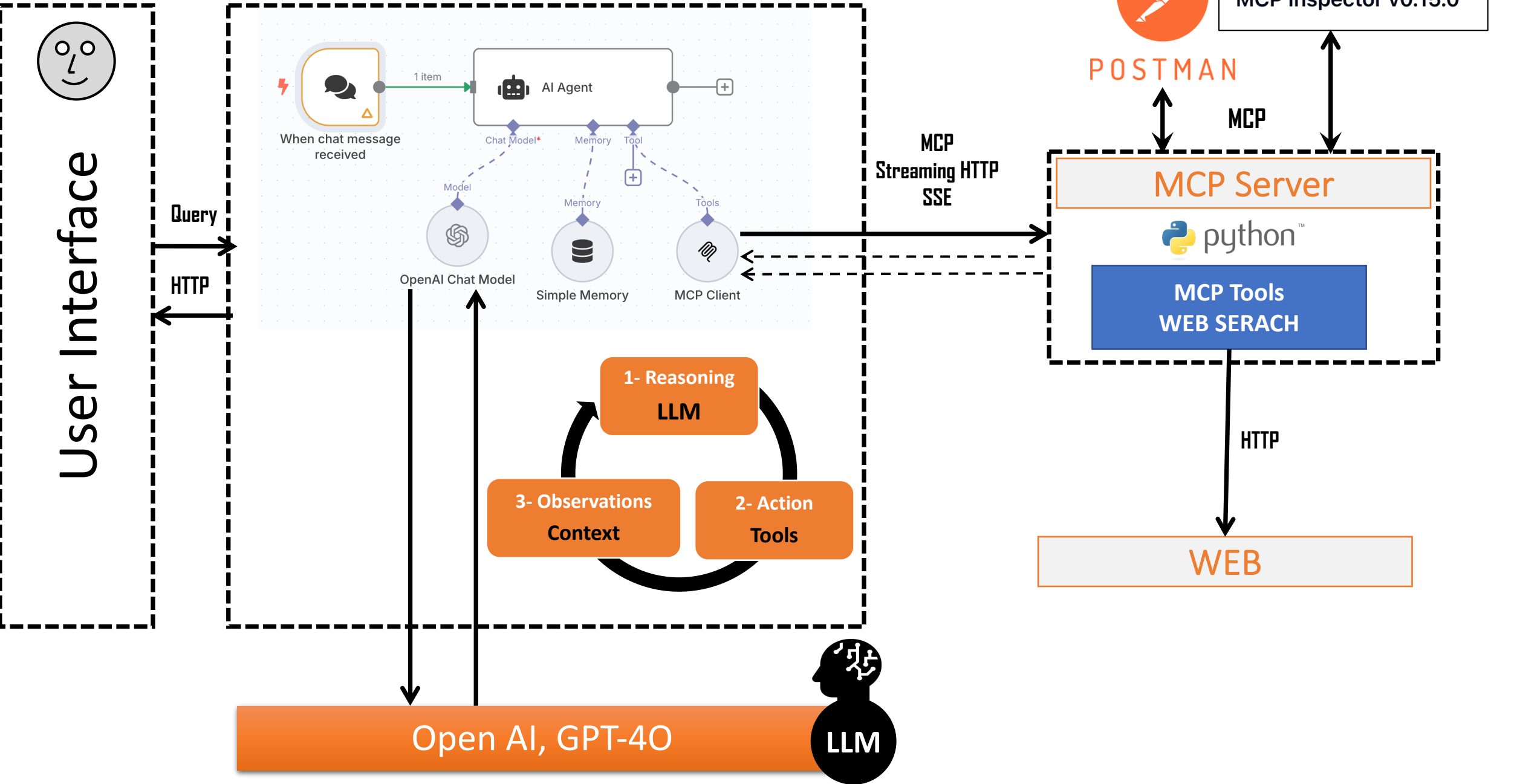
Architecture



Architecture



MCP : Architecture

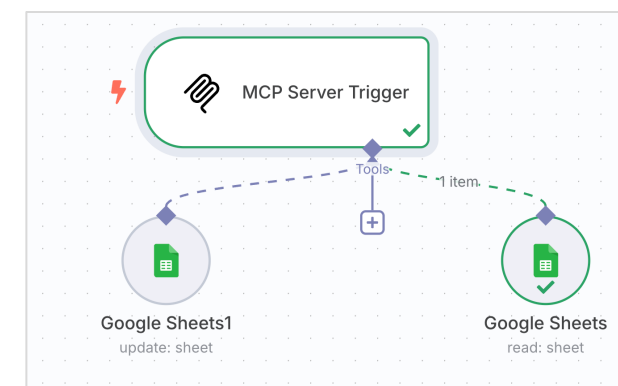
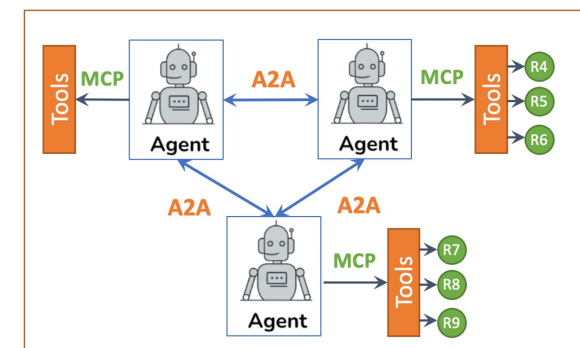
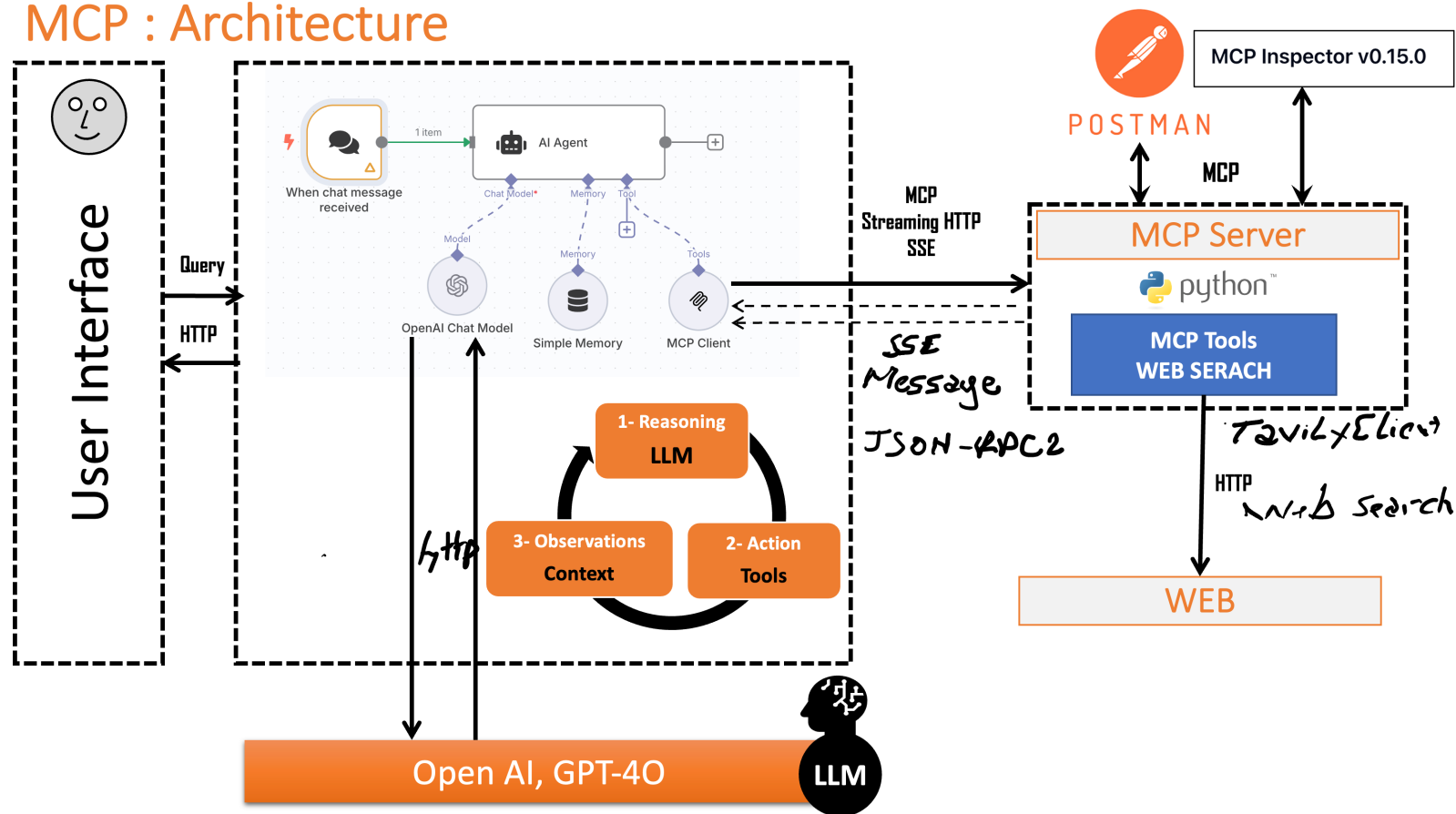


Remote MCP Server – AI Agent – N8N

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité

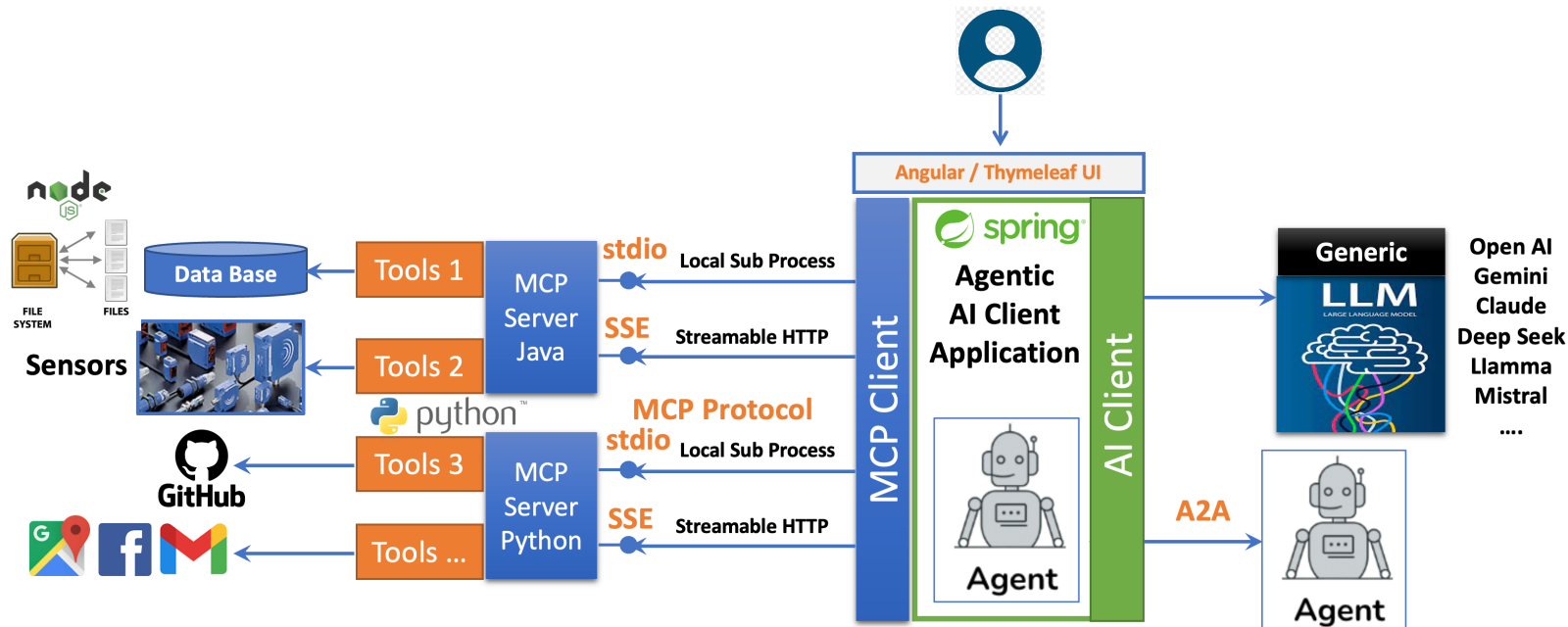


MCP : Architecture



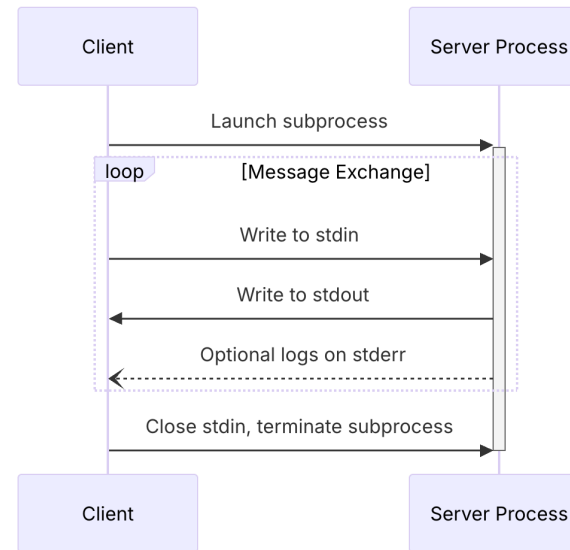
MCP

- Model Context Protocol (MCP) is an open protocol that enables seamless integration between LLM applications and external data sources and tools.
- MCP provides a standardized way for applications to:
 - Share contextual information with language models
 - Expose tools and capabilities to AI systems
 - Build composable integrations and workflows
- The protocol uses **JSON-RPC 2.0 messages** to establish communication between:
 - **Hosts:** LLM applications that initiate connections
 - **Clients:** Connectors within the host application
 - **Servers:** Services that provide context, tools and capabilities
- MCP standardizes how to integrate **additional context and tools** into the ecosystem of AI applications.

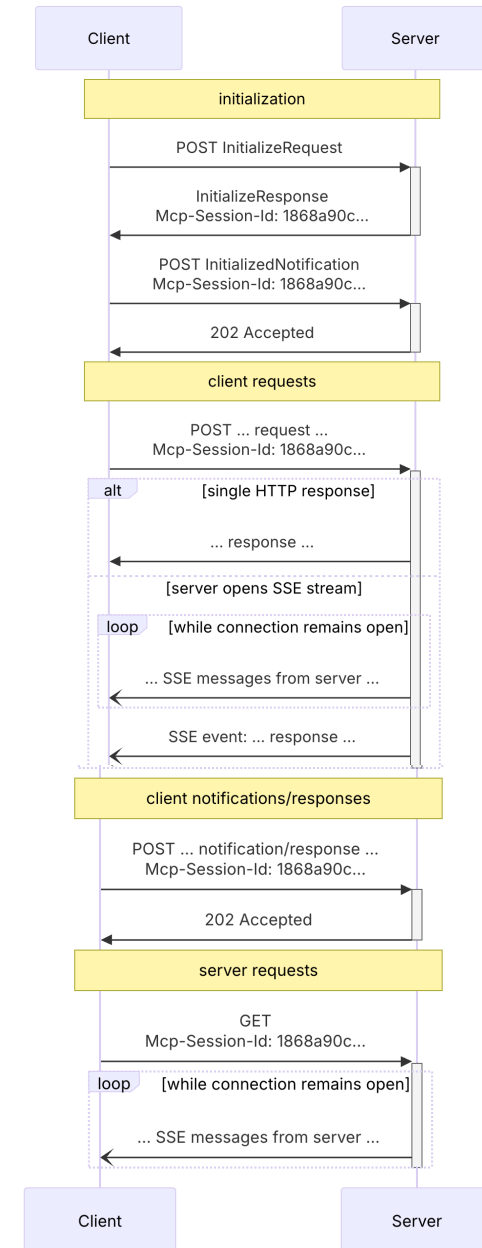


MCP : Transport (stdio)

- The protocol currently defines two standard transport mechanisms for client-server communication:
 - **stdio**, communication over **standard in** and **standard out**
 - **Streamable HTTP (SSE)**
- **STDIO**
 - In the **stdio** transport:
 - The client launches the MCP server as a subprocess.
 - The server reads JSON-RPC messages from its standard input (stdin) and sends messages to its standard output (stdout).
- **Streamable HTTP (SSE)** :
 - In the **Streamable HTTP** transport, the server operates as an independent process that can handle multiple client connections.
 - This transport uses HTTP POST and GET requests.
 - Server can optionally make use of **Server-Sent Events (SSE)** to stream multiple server messages.
 - The server **MUST** provide a single HTTP endpoint path that supports both POST and GET methods. For example, this could be a URL like <https://example.com/mcp>.



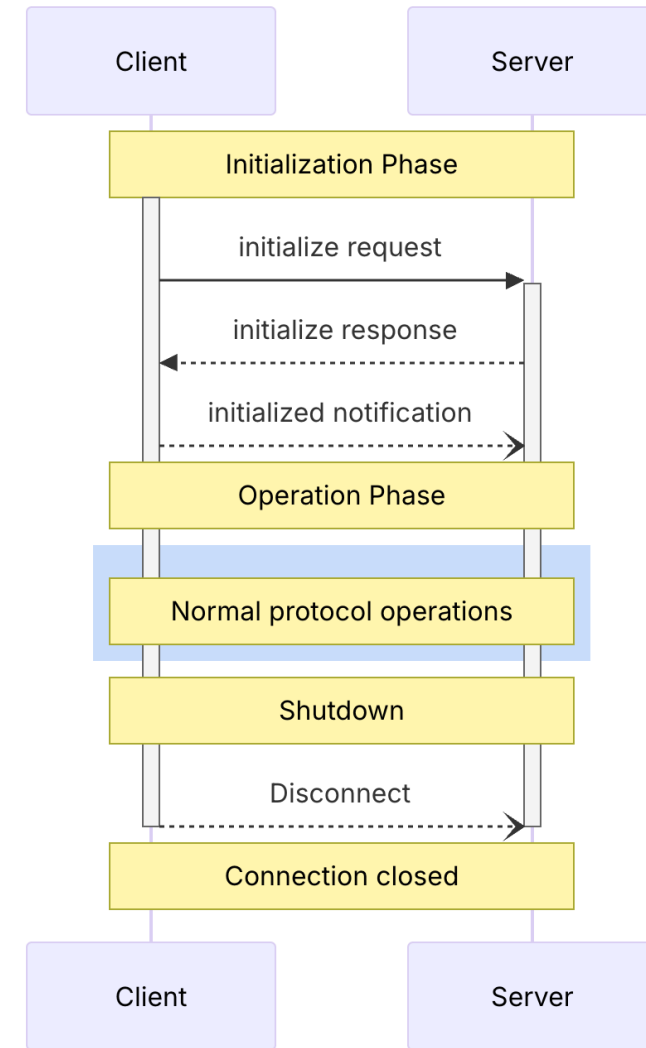
Sequence Diagram



MCP Lifecycle

<https://modelcontextprotocol.io/specification/2025-03-26/basic/lifecycle>

- **Base Protocol :**
 - JSON-RPC message format
 - Stateful connections
 - Server and client capability negotiation
- **Features :**
 - Servers offer any of the following features to clients:
 - **Resources:** Context and data, for the user or the AI model to use
 - **Prompts:** Templated messages and workflows for users
 - **Tools:** Functions for the AI model to execute
- **MCP Lifecycle:**
 - **Initialization:** Capability negotiation and protocol version agreement
 - **Operation:** Normal protocol communication
 - **Shutdown:** Graceful termination of the connection



MCP Lifecycle : Connection

GET

http://localhost:8089/sse

Close

ParamsAuthorizationHeaders (7)BodyScriptsTestsSettings

Query Params

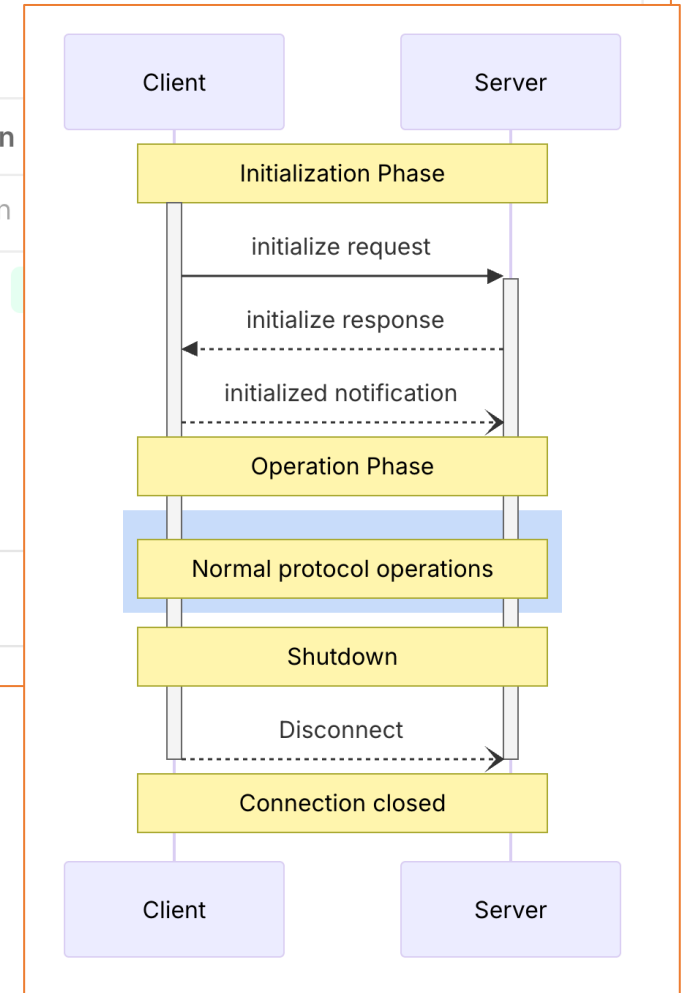
	Key	Value	Description
	Key	Value	Description

BodyCookiesHeaders (6)Test Results

Search

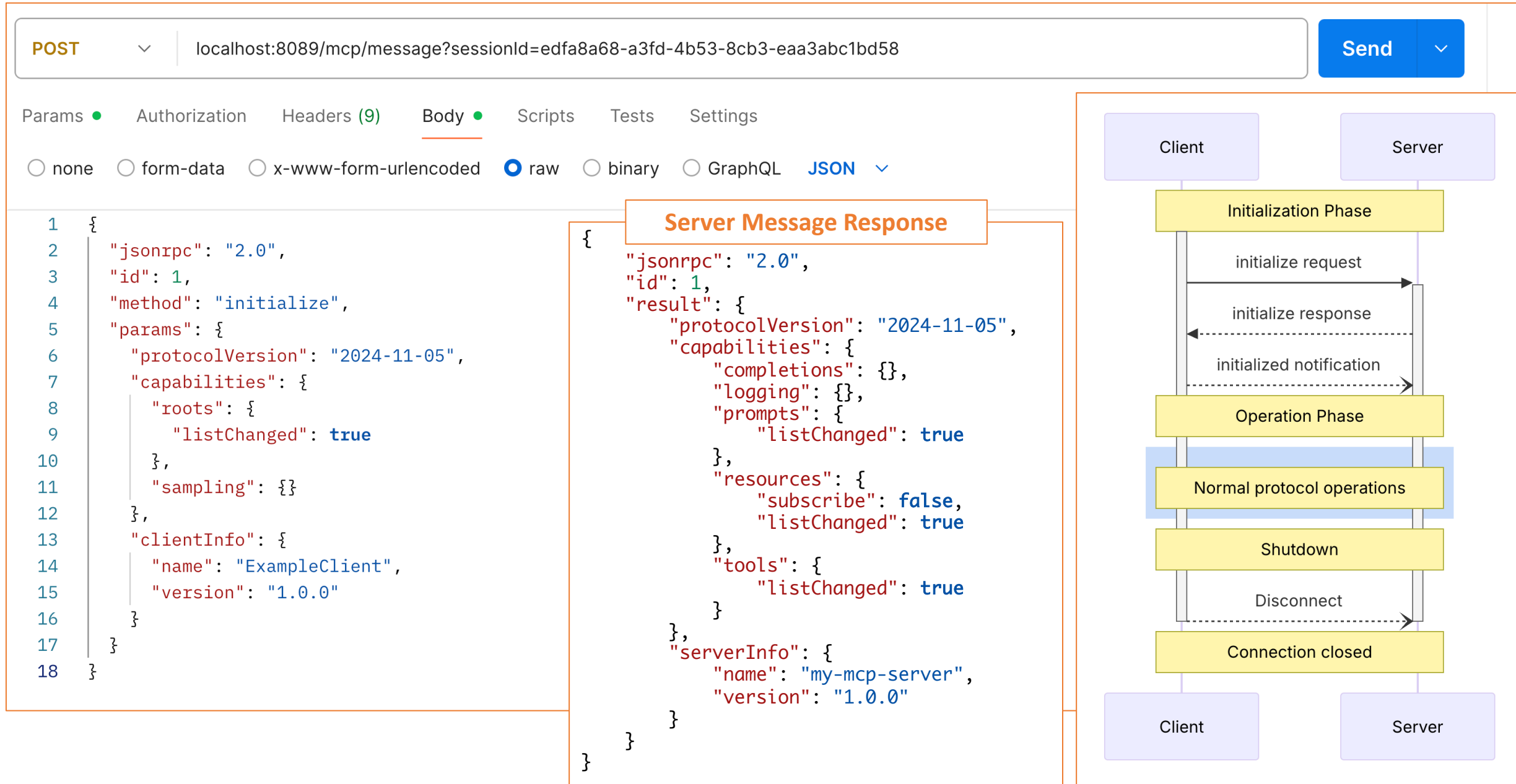
endpoint/mcp/message?sessionId=edfa8a68-a3fd-4b53-8cb3-eea3abc1bd58

Connected to http://localhost:8089/sse



```
$ npx @modelcontextprotocol/inspector
```

MCP Lifecycle : Initialization request



MCP Lifecycle : Initialization notification

POST

localhost:8089/mcp/message?sessionId=479a49ec-914f-4be8-9d61-93634e3c6a85

Send

Params

Authorization

Headers (9)

Body

Scripts

Tests

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

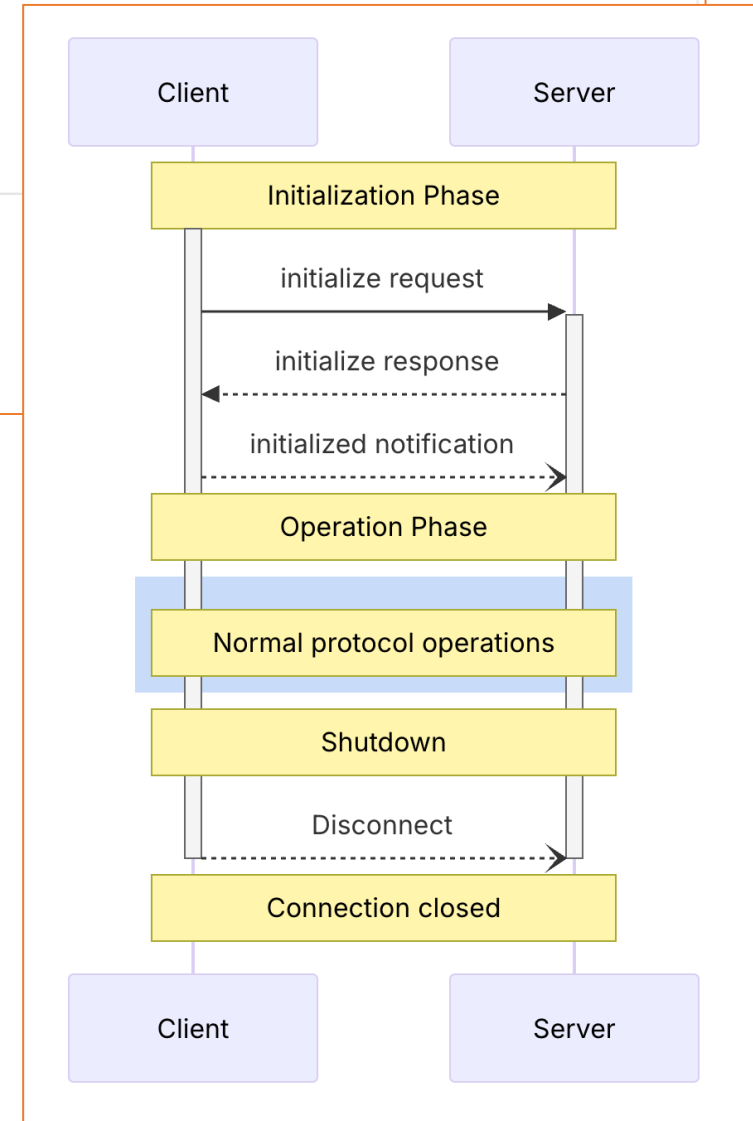
☒ raw

☐ binary

☐ GraphQL

JSON

```
1 {
2   "jsonrpc": "2.0",
3   "method": "notifications/initialized"
4 }
```



MCP Lifecycle : Operation phase (tools/list)

POST

localhost:8089/mcp/message?sessionId=479a49ec-914f-4be8-9d61-93634e3c6a85

Send

Params

Authorization

Headers (9)

Body

Scripts

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

{

2

"jsonrpc": "2.0",

3

"method": "tools/list",

4

"id": 1

5

}

Body

Cookies

Headers (4)

Test Results

Raw

Preview

Visualize

Server Message Response

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "multiplyNumbers",
        "description": "Multiply two Numbers",
        "inputSchema": {
          "type": "object",
          "properties": {
            "n1": {
              "type": "number",
              "format": "double"
            },
            "n2": {
              "type": "number",
              "format": "double"
            }
          },
          "required": [
            "n1",
            "n2"
          ],
          "additionalProperties": false
        }
      }
    ]
  }
}
```

Client

Server

Initialization Phase

"name": "addNumbers",

"description": "Add two Numbers",

"inputSchema": {

"type": "object",

"properties": {

"n1": {

"type": "number",

"format": "double"

},

"n2": {

"type": "number",

"format": "double"

}

},

"required": [

"n1",

"n2"

],

"additionalProperties": false

}

}

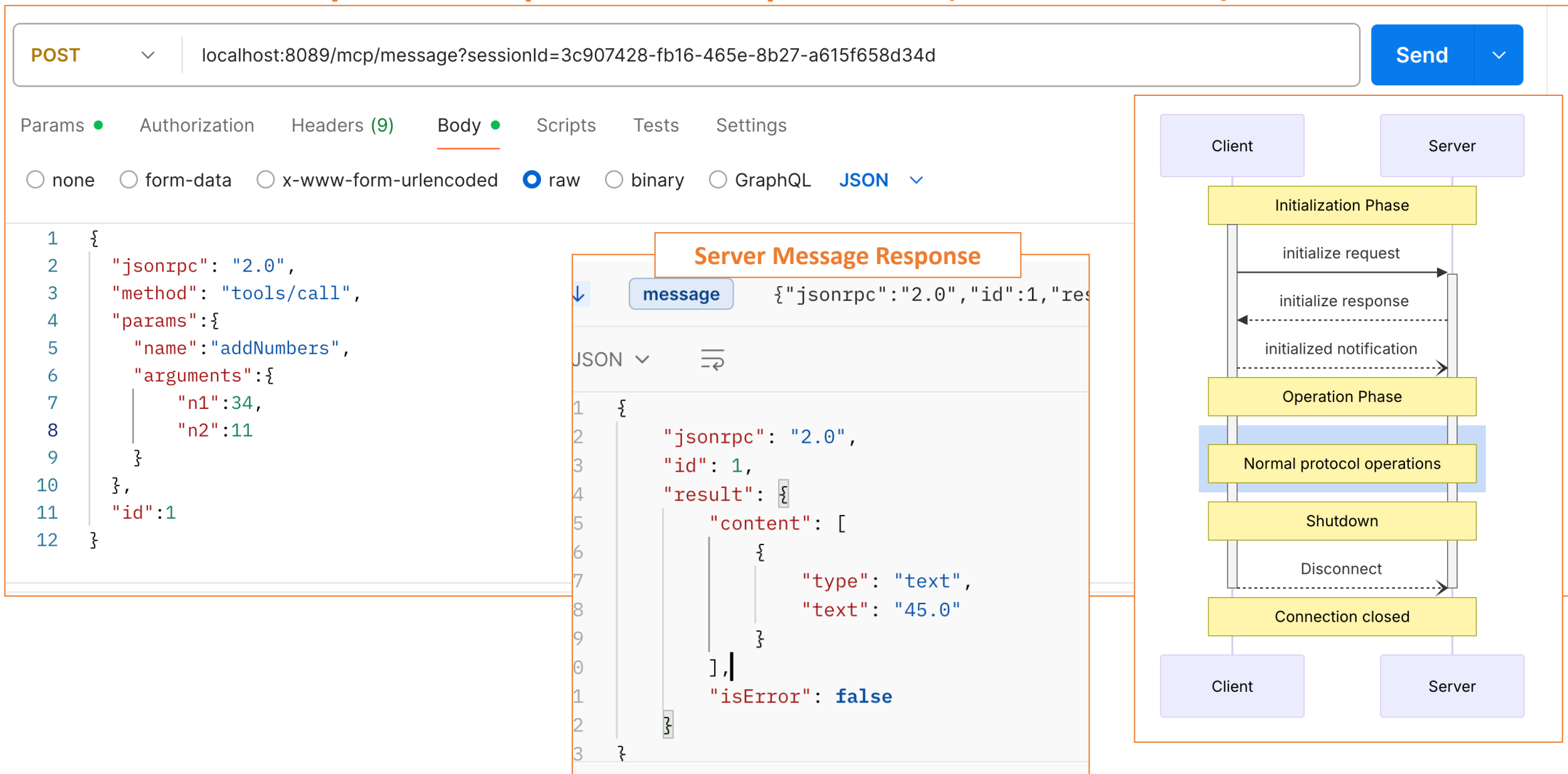
}

}

Client

Server

MCP Lifecycle : Operation phase (tools/call)



MCP Lifecycle : Operation phase (tools/call

Untitled MCP Request

SaveShare

HTTPlocalhost:8089/sseRun

MessageAuthorizationHeadersSettings

ToolsPromptsResources

multiplyNumbers

Multiply two Numbers

n1*: number

78

n2*: number

12

2. addNumbers

Add two Numbers

ResponseNotifications

SearchAll Messages

Connected

23:12:16.076

Server Message Response

```
1 {
2   "method": "tools/call",
3   "params": {
4     "name": "multiplyNumbers",
5     "arguments": {
6       "n1": 78,
7       "n2": 12
8     }
9   }
10 }
```

ClientServer

Initialization Phase

initialize request

initialize response

initialized notification

Operation Phase

Normal protocol operations

Shutdown

Disconnect

Connection closed

ClientServer

AI Agents Frameworks

- **OpenAI Agent SDK** : <https://openai.github.io/openai-agents-python/>
- **CrewAI** : <https://docs.crewai.com/introduction>
- **LangChain** : <https://python.langchain.com/docs/tutorials/agents/>
- **AutoGen** : <https://microsoft.github.io/autogen/dev//user-guide/agentchat-user-guide/tutorial/agents.html>
- **Pydantic AI Agent** : <https://ai.pydantic.dev/>
- **Phidata** : <https://docs.phidata.com/introduction>
- **PraisonAiAgents** : <https://docs.praison.ai/framework/praisonaigents>
- ...

OpenAI Agents SDK

- The OpenAI Agents SDK aims to build agentic AI apps in a lightweight, easy-to-use package with very few abstractions.
- It's a production-ready upgrade of our previous experimentation for agents, Swarm.
- The Agents SDK has a very small set of primitives:
 - **Agents**, which are LLMs equipped with instructions and tools
 - **Handoffs**, which allow agents to delegate to other agents for specific tasks
 - **Guardrails**, which enable the inputs to agents to be validated
 - Support **Model Context Protocol (MCP)**

Installation

```
pip install openai-agents
```

Hello world example

```
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="You are a helpful assistant")

result = Runner.run_sync(agent, "Write a haiku about recursion in programming.")
print(result.final_output)

# Code within the code,
# Functions calling themselves,
# Infinite loop's dance.
```

```
export OPENAI_API_KEY=sk-...
```

DEMO : Agentic AI

Source code and resources : <https://github.com/mohamedYoussfi/agentic-ai>

```
> pip install uv
> uv init
> uv venv
> source .venv/bin/activate (MACOS)
> .venv\Scripts\activate.bat (Windows)
> uv add ipykernel
> uv add "openai-agents[viz]"
> uv add pydantic
> uv add python-dotenv
> uv add streamlit
```

• pyproject.toml

```
[project]
name = "test"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "duckduckgo-search>=8.0.1",
    "ipykernel>=6.29.5",
    "openai-agents[viz]>=0.0.14",
    "pydantic>=2.11.4",
    "python-dotenv>=1.1.0",
    "streamlit>=1.45.0",
]
```

First Agent Using OpenAI Model

```
from dotenv import load_dotenv
import os
from IPython.display import Markdown
```

✓ 0.0s

```
load_dotenv()
```

✓ 0.0s

```
api_key = os.environ.get('OPENAI_API_KEY')
if not api_key:
    raise("API KEY not available")
```

✓ 0.0s



```
OPENAI_API_KEY=sk-proj-...
```

```
from agents import Agent, Runner, OpenAIChatCompletionsModel, AsyncOpenAI
```

✓ 0.5s

```
cost_medico_agent = Agent(
    name="Doctor Agent",
    model="gpt-4o",
    instructions="""
    Donne moi des conseils concernant la pathologie fournie par l'utilisateur
    """
)
result = await(Runner.run(cost_medico_agent,"Cardiologie"))
```

```
display(Markdown(result.final_output))
```

En ce qui concerne la cardiologie, il est important de prendre soin de la santé cardiovasculaire en suivant plusieurs conseils généraux:

1. **Alimentation équilibrée** : Adopte un régime riche en fruits, légumes, grains entiers, protéines maigres et acides gras oméga-3 (présents dans le poisson, les noix et les graines).
2. **Activité physique régulière** : Vise au moins 150 minutes d'exercice modéré par semaine, comme la marche rapide, la natation ou le vélo.
3. **Gestion du stress** : Pratiques comme le yoga, la méditation ou des exercices de respiration peuvent aider à réduire le stress.
4. **Arrêt du tabac** : Si tu fumes, chercher de l'aide pour arrêter peut drastiquement améliorer ta santé cardiaque.
5. **Contrôle de la pression artérielle et du cholestérol** : Effectue des contrôles réguliers et prends les mesures nécessaires si les niveaux sont élevés.
6. **Maintien d'un poids sain** : Un poids adéquat peut réduire le risque de maladies cardiovasculaires.
7. **Modération de la consommation d'alcool** : Limite ta consommation d'alcool si tu choisis de boire.
8. **Suivi médical** : Effectue des bilans de santé réguliers pour détecter précocement les problèmes.

Si tu as des soucis spécifiques ou un diagnostic, il est important de consulter un cardiologue pour un plan de traitement adapté à ton cas.

First Agent Using OpenAI Llama 3.2 Model

```
llama_model = OpenAIChatCompletionsModel(  
    ... model="llama3.2",  
    ... openai_client= AsyncOpenAI(base_url="http://localhost:11434/v1")  
)
```

```
free_medico_agent = Agent(  
    name="Doctor Agent",  
    model=llama_model,  
    instructions="""  
    Donne moi des conseils concernant la pathologie fournie par l'utilisateur  
    """"  
)  
  
result = await(Runner.run(free_medico_agent,"Cardiologie"))
```

```
display(Markdown(result.final_output))
```

Voici quelques informations générales et conseils relatifs à la cardiologie :

Qu'est-ce que le cœur ?

Le cœur est un organe essentiel de notre corps qui frappe sur environ 100 000 fois par jour. Il est responsable de pousser l'oxygène à travers les artères et d'amener à retourner au cœur le déchet, comme les déchets du cerveau. Le cœur est établi à la base du torso.

Les troubles cardiaques

Il existe une large gamme de maladies cardiovasculaires qui affectent millions de personnes dans le monde entier. Voici quelques exemples :

1. **Insuffisance cardiaque**: Lorsqu'elles ne fonctionnent pas avec suffisamment pour fournir les doses nécessaires à tout le corps, les organes peuvent s'empirer.
2. **Coronaropathie** : Cette maladie est due au stress des artères. Tout ce que vous avez besoin de faire, c'est que votre système d'ongulats réagisse très bien avec une pression.
3. **Aortique et pulmonaire hypertension**: le risque de stent ou d'assister à un cedran
4. **Arérodite hémorragique** : sache qu'il y a peut-être des problèmes en cas d'hémorragie
5. **Anomalies du cœur (AESC)**

```
mohamedyoussefi — fina@aifinashore: ~ — ollama run llama3.2 — 74x16  
Last login: Sun May 11 08:29:31 on ttys027  
(base) $ ollama ls  
NAME                                ID                                SIZE    MODIFIED  
llama3.2:latest                     a80c4f17acd5                     2.0 GB  2 days ago  
deepseek-r1:7b                      0a8c26691023                     4.7 GB  3 months ago  
mxbai-embed-large:latest            468836162de7                     669 MB  5 months ago  
llama3.2-vision:latest              38107a0cd119                     7.9 GB  6 months ago  
llama3.1:latest                     62757c860e01                     4.7 GB  9 months ago  
llama2:latest                       78e26419b446                     3.8 GB  10 months ago  
llava:latest                        8dd30f6b0cb1                     4.7 GB  11 months ago  
llama3:latest                       71a106a91016                     4.7 GB  12 months ago  
mistral:latest                      61e88e884507                     4.1 GB  13 months ago  
(base) $ ollama run llama3.2  
>>> end a message (/? for help)
```

Agent and Structured Output

```
from pydantic import BaseModel
```

```
class Recipe(BaseModel):  
    title : str  
    ingredients : list[str]  
    cooking_time : int  
    servings : int
```

```
recipe_agent = Agent(  
    name = "Recipe Agent",  
    model="gpt-4o",  
    instructions= """  
You are an agent for creating Recipes  
You will be given the name of the food to create  
Your job is to create this food as an actual detailed recipe  
The cooking time should be in minutes  
""",  
    output_type= Recipe  
)
```

```
response = await Runner.run(recipe_agent,"Tajine Marocain en Arabe")  
recipe = response.final_output
```

```
print("*"*40)  
print(f>Title : {recipe.title}")  
print(f>Time : {recipe.cooking_time} minutes")  
print(f>Servings : {recipe.servings}")  
for ing in recipe.ingredients:  
    print(f>- {ing}")
```

Title : طاجين مغربي

Time : 120 minutes

Servings : 4

- كغ من لحم الغنم (أو الدجاج حسب الرغبة) 1
- بصلات متوسطة الحجم، مقطعة إلى شرائح 2
- فصوص ثوم مفرومة 3
- ملاعق كبيرة من زيت الزيتون 4
- ملعقة صغيرة من الزنجبيل المطحون 1
- ملعقة صغيرة من الكركم 1
- ملعقة صغيرة من القرفة 1
- ملح وفلفل حسب الرغبة
- غرام من الزبيب 100
- غرام من اللوز المقلي 100
- عصير نصف ليمونة
- كوب من الماء 2
- حفنة من الكزبرة الطازجة، مفرومة للزينة
- غرام من الخضروات الموسمية (مثل الجزر والبطاطس)، مقطعة 200

Agent and Tools

```
from agents import Agent, function_tool, WebSearchTool
```

```
@function_tool
```

```
def get_weather(city : str) -> str:
```

```
    """
```

```
    Get the weather of a given city
```

```
    """
```

```
    print(f"The tool get_weather is used by the agent to get the weather of the {city}")
```

```
    return f"The weather of {city} is sunny"
```

```
@function_tool
```

```
def get_temperature(city : str) -> str:
```

```
    """
```

```
    Get the temperature in a given city
```

```
    """
```

```
    print(f"The tool get_temperature is used by the agent to get the temperature of the {city}")
```

```
    return f"it is 50° In {city}"
```

```
weather_agent = Agent (
```

```
    name="The Weather Agent",
```

```
    instructions="""
```

```
    As the local weather agent, you will be asked to give information  
    about weather of a given city,
```

```
    Your output should include weather information asked by the user"
```

```
    """
```

```
    tools=[get_weather, get_temperature]
```

```
)
```

```
response = await Runner.run(weather_agent, "The temperature and weather in Marrakech")  
response.final_output
```

The tool get_weather is used by the agent to get the weather of the Marrakech
The tool get_temperature is used by the agent to get the temperature of the Ma

'The weather in Marrakech is sunny, with a temperature of 50°F.'

```
response = await Runner.run(weather_agent, "Quel temps fera t-il demain à Tanger")  
response.final_output
```

The tool get_weather is used by the agent to get the weather of the Tanger

'Demain à Tanger, le temps sera ensoleillé. ☀'

Agent and Tools

```
search_agent = Agent(  
    ... name="Search Agent",  
    ... instructions="""  
    ... You are a news reporter,  
    ... your job is to search new recent articles using internet about a given country  
    ... """,  
    ... tools=[WebSearchTool()]  
)
```

```
from duckduckgo_search import DDGS  
from datetime import datetime
```

```
now = datetime.now().strftime("%Y-%m-%d")  
now
```

```
@function_tool  
def get_news_articles(topic:str):  
    print(f"Running DuckDuckSearch for news in topic : {topic}")  
    ddgs_api = DDGS()  
    results = ddgs_api.text(f"{topic}, {now}", max_results=5)  
    return results
```

```
response = await Runner.run(search_agent, "Les actualités au Maroc")
```

```
display(Markdown(response.final_output))
```

```
search_agent = Agent(  
    name= "Search Agent",  
    instructions= """  
    You are a news reporter,  
    your job is to search new recent articles using internet about a given country  
    """,  
    tools=[get_news_articles]  
)
```

Running DuckDuckSearch for news in topic : المغرب

```
display(Markdown(response.final_output))
```

✓ 0.0s

آخر الأخبار حول المغرب

- حكومة بيدرو سانثيز تمنح المغرب 340 مليون يورو لبناء مشروع محطة تحلية المياه.
الرابط: [اقرأ المزيد](#)
...مقتطف: حكومة بيدرو سانثيز منحت المغرب مبلغ 340 مليون يورو لتمويل مشروع محطة لتحلية المياه
- التقويم والأعياد الرسمية للمغرب سنة 2025.
الرابط: [اقرأ المزيد](#)
...مقتطف: تشمل العطل الرسمية في المغرب عام 2025 رأس السنة الجديدة وذكرى بيان الاستقلال
- المملكة المغربية لسنة 2025 - تفاصيل العطل

Agent and Handoffs

```
from IPython.display import Markdown
from agents.extensions.visualization import draw_graph
```

```
from agents import Agent
from pydantic import BaseModel
```

```
class Tutorial(BaseModel):
    outline : str
    tutorial : str
```

```
tutorial_response = await Runner.run(outline_builder_agent,
                                     "Boucles en Java avec des explications en Français")
tuto = tutorial_response.final_output
```

```
tutorial_generator_agent = Agent(
    name="Tutorial Generator Agent",
    handoff_description= "Used to generate tutorial for a given outline",
    instructions="""
        Given a programming topic and an outline,
        your job is to generate code snippets for each section of the outline,
        Format the tutorial in Markdown using a mix of text explanation and code snippets examples",
        Include comments in the code snippets to further explain the code"
    """,
    output_type=Tutorial
)
```

```
display(Markdown("### Outline :"))
display(Markdown(tuto.outline))
print("*"*60)
display(Markdown("### Totorial :"))
display(Markdown(tutorial_response.final_output.tutorial))
```

```
draw_graph(outline_builder_agent)
```

```
outline_builder_agent = Agent(
    name= "Outline builder agent",
    instructions="""
        Given a particular programming topic, your job is to generate a tutorial by crafting an outline
        After making the outline, hand it to the tutorial generator agent"
    """,
    handoffs=[tutorial_generator_agent]
)
```

Outline :

1. Introduction

- Pour
- Type

2. La boucle

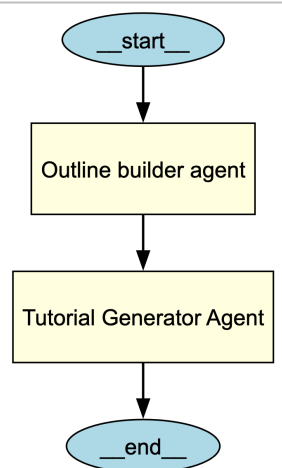
- Syn
- Exe
- Util

3. La boucle

- Syn
- Exe
- Cor

4. La boucle

- Syn
- Exemple simple
- Différences avec `while`



Triage Agent

```
from agents import Agent, handoff, RunContextWrapper, function_tool
from agents.extensions.visualization import draw_graph
```

```
@function_tool
def get_teacher_name(course : str):
    """
    Get The teacher name if course (Mathematics, History)
    """
    (parameter) course: str
    if(course=='Mathe
        return "Pr. H
    else:
        return "Pr. Najlaa"
```

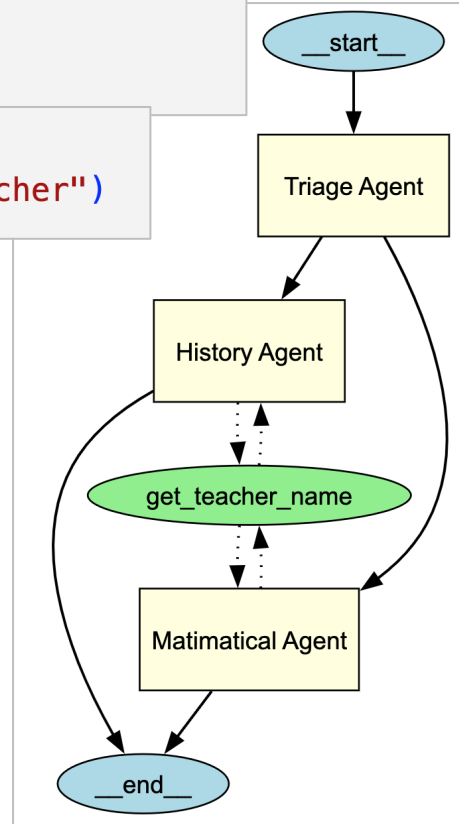
```
trriage_agent = Agent(
    name= "Triage Agent",
    instructions="""
        Answer the user question by delegating the task to the aproppriate agent,
        If neither agent is relevant, give a general response
    """,
    handoffs=[history_agent, math_agent]
)
```

```
trial_response = await Runner.run(trriage_agent,
    "How do i multiply X by y and give me the name of teacher")
```

```
history_agent = Agent(
    name= "History Agent",
    handoff_description= "Specialist Agent for historical questions",
    instructions="""
        You provide asistant with the historical queries
        Explain important events and context clearly
    """,
    tools= [get_teacher_name]
)
```

```
math_agent = Agent(
    name= "Matimatical Agent",
    handoff_description= "Specialist Agent for matimatical questions",
    instructions= """
        You provide asistant with the matimatical queries
        Explain your reasoning at each step and include examples
    """,
    tools= [get_teacher_name]
)
```

```
draw_graph(trriage_agent)
```



Triage Agent

```
from agents import Agent, handoff, RunContextWrapper, function_tool
from agents.extensions.visualization import draw_graph
```

```
@function_tool
def get_teacher_name(course : str):
    """
    Get The teacher name if course (Mathematics, History)
    """
    (parameter) course: str
    if(course=='Mathe
        return "Pr. H
    else:
        return "Pr. Najlaa"
```

```
triage_agent = Agent(
    name= "Triage Agent",
    instructions="""
        Answer the user question by delegating the task to the aproppriate agent,
        If neither agent is relevant, give a general response
    """,
    handoffs=[history_agent, math_agent]
)
```

```
trial_response = await Runner.run(triage_agent,
    "How do i multiply X by y and give me the name of teacher")
```

```
history_agent = Agent(
    name= "History Agent",
    handoff_description= "Specialist Agent for historical questions",
    instructions="""
        You provide asistant with the historical queries
        Explain important events and context clearlly
    """,
    tools= [get_teacher_name]
)
```

```
math_agent = Agent(
    name= "Matimatical Agent",
    handoff_description= "Specialist Agent for matimatical questions",
    instructions= """
        You provide asistant with the matimatical queries
        Explain your reasoning at each step and include examples
    """,
    tools= [get_teacher_name]
)
```

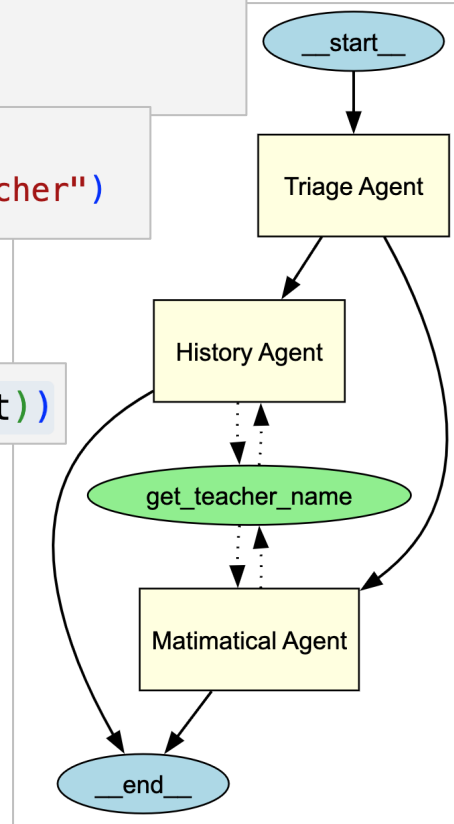
```
draw_graph(triage_agent)
```

```
display(Markdown(trial_response.final_output))
```

To multiply two numbers, (x) and (y), follow these steps:

- Understand the Multiplication:** Multiplication is a shortcut for repeated addition.
- Perform the Multiplication:**
 - If both (x) and (y) are positive integers, multiply them directly.
 - For example, if (x = 3) and (y = 4), then (3 \times 4 = 12).
- Use a Calculator for Complex Numbers:**
 - For large or decimal numbers, you might want to use a calculator.
- Special Properties:**
 - Zero Property:** Any number multiplied by 0 is 0.
 - E.g., (5 \times 0 = 0).
 - Identity Property:** Any number multiplied by 1 remains unchanged.
 - E.g., (7 \times 1 = 7).

The Mathematics teacher is Pr. Hassan.



Triage Agent

```
def on_history_handoff(ctx : RunContextWrapper[None]):
    print ("="*60)
    print ("History Agent is Invoqued")
    print ("="*60)

def on_math_handoff(ctx : RunContextWrapper[None]):
    print ("="*60)
    print ("Math Agent is Invoqued")
    print ("="*60)
```

```
trriage_agent = Agent(
    name= "Triage Agent",
    instructions="""
        Answer the user question by delegating the task to the apropriate agent,
        If neither agent is relevant, give a general response
    """,
    handoffs=[
        handoff(history_agent, on_handoff=on_history_handoff),
        handoff(math_agent, on_handoff=on_math_handoff)
    ]
)
```

```
trial_response = await Runner.run(trriage_agent,"HISTORY OF Rabat")
```

=====

History Agent is Invoqued

=====

```
trial_response = await Runner.run(trriage_agent,"C'est quoi l'ENSET")
display(Markdown(trial_response.final_output))
```

L'ENSET (École Normale Supérieure de l'Enseignement des disciplines techniques et professionnelles. Elle est chargée de former les futurs enseignants à transmettre des connaissances et les programmes peuvent inclure des études théoriques

```
trial_response = await Runner.run(trriage_agent,"How do i multiply X by y")
```

```
*****
Math Agent is Invoqued
*****
```

Agent Traces

<https://platform.openai.com/traces>

```
from agents import trace
```

```
with trace("Triage Work Flow"):
    trial_response = await Runner.run(triage_agent, "Hitoire de Rabat")
    trial_response.final_output
```

```
=====
History Agent is Invoqued
=====
```

< Traces / MCP Filesystem Example  trace_71e50a1f348640...			 Refresh	
Assistant			10.82 s	
List MCP Tools			3 ms	
POST /v1/responses			3,422 ms	
write_file			2 ms	
POST /v1/responses			1,361 ms	
list_allowed_directories			2 ms	
POST /v1/responses			2,944 ms	
write_file			3 ms	
POST /v1/responses			3,082 ms	
Properties				
ID			trace_71e50a1f348640dd910e454a2...	
Workflow name			MCP Filesystem Example	
Metadata				
No metadata entries				

Agent Streaming

```
from openai.types.responses import ResponseTextDeltaEvent
joke_agent = Agent(
    name="My Agent",
    model='gpt-4o',
    instructions="You are a joke teller. you are giving a topic and you will tell 10 jokes about it"
)
result = Runner.run_streamed(joke_agent, "Food")
async for event in result.stream_events():
    if event.type=='raw_response_event' and isinstance(event.data, ResponseTextDeltaEvent):
        print(event.data.delta, end="", flush=True)
```

Sure thing! Here are 10 food jokes for you:

1. Why did the tomato turn red?
Because it saw the salad dressing!
2. What did the taco say to the burrito?
"I'm nacho friend anymore!"
3. Why don't eggs tell jokes?
They might crack up!
4. What do you call cheese that's not yours?
Nacho cheese!

Agent MCP : Model Context Protocol

```
## Create agent_mcp.py File
## Then Execute this application in console : > python
agent_mcp.py
import asyncio
import os
import shutil
from agents import Agent, Runner, gen_trace_id, trace
from agents.mcp import MCPServer, MCPServerStdio
from dotenv import load_dotenv

load_dotenv()

async def run(mcp_server: MCPServer):
    agent = Agent(
        name="Assistant",
        instructions="Use the tools to read the filesystem and
answer questions based on those files.",
        mcp_servers=[mcp_server],
    )

    # List the files it can read
    message = "Read the files and list them."
    print(f"Running: {message}")
    result = await Runner.run(starting_agent=agent, input=message)
    print(result.final_output)
```

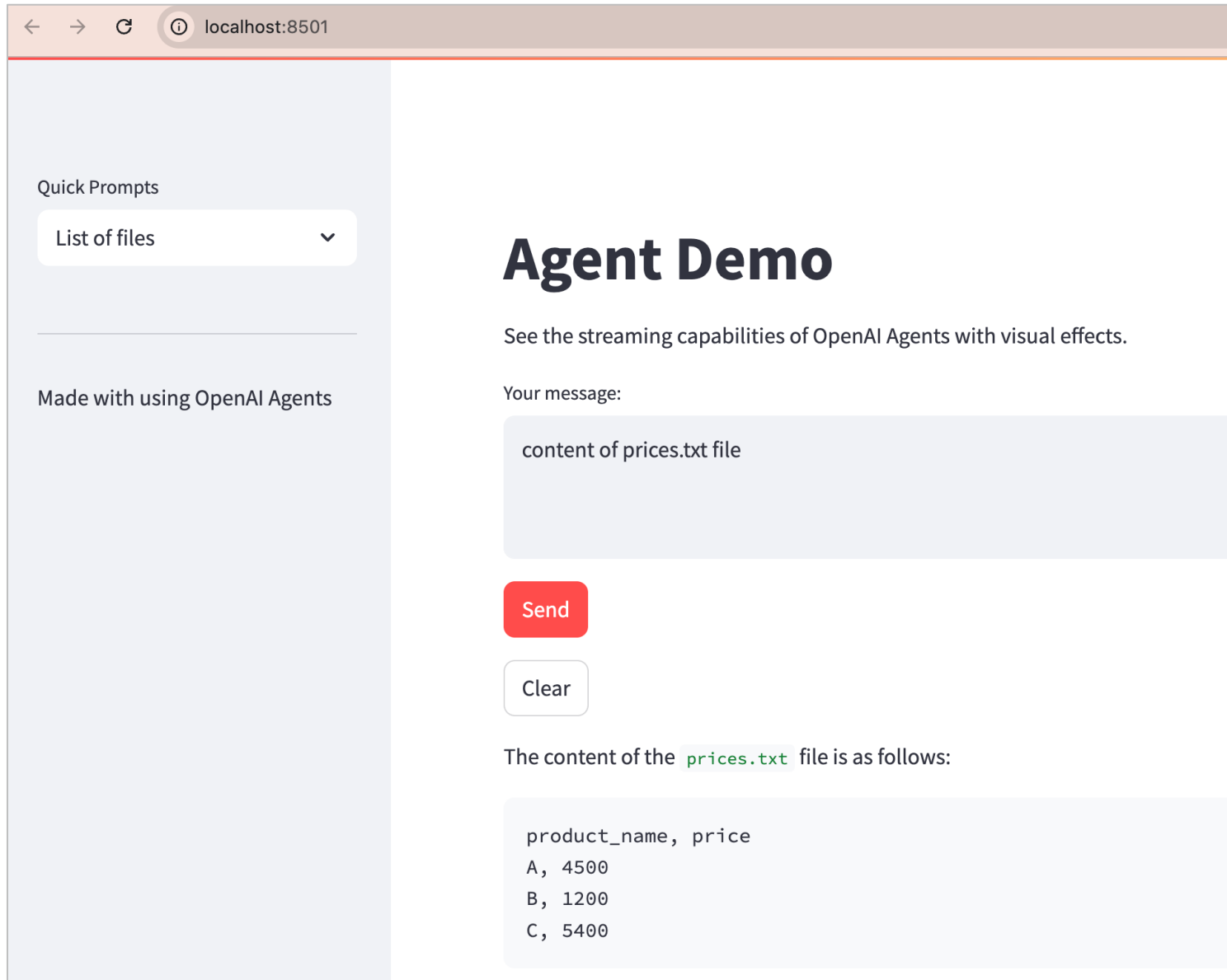
```
# Ask about books
message = "What is my #1 favorite book?"
print(f"\n\nRunning: {message}")
result = await Runner.run(starting_agent=agent, input=message)
print(result.final_output)

# Ask a question that reads then reasons.
message = "Look at my favorite songs. Suggest one new song that I might
like."
print(f"\n\nRunning: {message}")
result = await Runner.run(starting_agent=agent, input=message)
print(result.final_output)

async def main():
    current_dir = os.path.dirname(os.path.abspath(__file__))
    samples_dir = os.path.join(current_dir, "sample_files")
    os.makedirs(samples_dir, exist_ok=True)
    async with MCPServerStdio(
        name="Filesystem Server, via npx",
        params={
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-filesystem"],
        },
        samples_dir,
    ) as server:
        trace_id = gen_trace_id()
        with trace(workflow_name="MCP Filesystem Example", trace_id=trace_id):
            print(
                f"View trace:
https://platform.openai.com/traces/trace?trace_id={trace_id}\n"
            )
            await run(server)

if __name__ == "__main__":
    # Let's make sure the user has npx installed
    if not shutil.which("npx"):
        raise RuntimeError(
            "npx is not installed. Please install it with `npm install -g npx`."
        )
    asyncio.run(main())
```

Agent MCP : Model Context Protocol with Streamlit UI



Agent MCP : Model Context Protocol with Streamlit UI

```
import asyncio
import streamlit as st
from openai.types.responses import ResponseTextDeltaEvent
from agents.mcp import MCPServer, MCPServerStdio, MCPServerSse
import sys
import os
from dotenv import load_dotenv
load_dotenv()
api_key = os.environ.get("OPENAI_API_KEY")
if not api_key:
    raise ("API KEY not available")
# Add the parent directory to the path so we can import the agents
module
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from agents import Agent, Runner
st.set_page_config(
    page_title="Agent Demo",
    page_icon=".",
    layout="wide",
)
st.title("Agent Demo")
st.write("See the streaming capabilities of OpenAI Agents with visual effects.")
```

```
with st.sidebar:
    demo_options = [
        "List of files",
        "if not created, create new file data.txt",
        "content of data.txt file",
        "Look at my favorite songs. Suggest one new song that I might like.",
    ]
    demo_prompt = st.selectbox("Quick Prompts", demo_options)
    st.markdown("----")
    st.markdown("Made with using OpenAI Agents")
# User input area
user_input = st.text_area("Your message:", value=demo_prompt, height=100)
send_button = st.button("Send", type="primary")
# Response area with custom styling
response_container = st.container()
message_placeholder = st.empty()
```

Agent MCP : Model Context Protocol with Streamlit UI

```
import asyncio
import os
import shutil

from agents import Agent, Runner, gen_trace_id, trace
from agents.mcp import MCPServer, MCPServerStdio
from dotenv import load_dotenv

load_dotenv()

async def run(mcp_server: MCPServer):
    agent = Agent(
        name="Assistant",
        instructions="answer the user question using provided tools",
        mcp_servers=[mcp_server],
    )

    if send_button and user_input:
        with response_container:
            with st.spinner("Thinking..."):
                result = await Runner.run(starting_agent=agent,
input=user_input)

                print(result.final_output)
                message_placeholder.markdown(result.final_output)

            # Add clear button
            if st.button("Clear"):
                st.experimental_rerun()
```

```
# Instructions
if not send_button:
    with response_container:
        st.info("👉 Enter your message above and click 'Send' to see the streaming response.")
        st.markdown(
            """
            ### Tips:
            - Choose from the quick prompts or enter your own
            """
        )

async def main():
    current_dir = os.path.dirname(os.path.abspath(__file__))
    samples_dir = os.path.join(current_dir, "sample_files")
    os.makedirs(samples_dir, exist_ok=True)
    async with MCPServerStdio(
        name="Filesystem Server, via npx",
        params={
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-filesystem", samples_dir],
        },
    ) as server:
        trace_id = gen_trace_id()
        with trace(workflow_name="MCP Filesystem Example", trace_id=trace_id):
            print(
                f"View trace: https://platform.openai.com/traces/trace?trace_id={trace_id}\n"
            )
            await run(server)

if __name__ == "__main__":
    # Let's make sure the user has npx installed
    if not shutil.which("npx"):
        raise RuntimeError(
            "npx is not installed. "
        )

    asyncio.run(main())
```



Data Analytics With Tableau Public

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité

